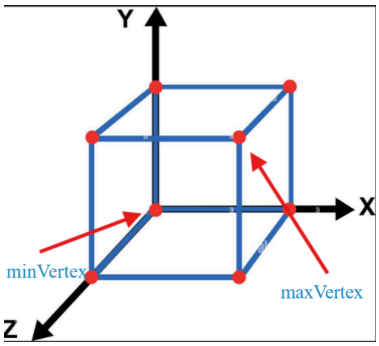
Nama : Wildan Abdurrahman Ghazali
NIM: 13524054

Laporan Tugas Kecil 2 IF2211 Strategi Algoritma : Voxelization Objek 3D menggunakan Octree

Program Utama

A. Struktur data Utama:

1. Vertex yang menyimpan koordinat x,y,z dari input file .obj
2. Face yang menyimpan segitiga yang terdiri dari 3 sudut vertex
3. Octree sebagai pohon utama, berisi :
   a. OctreeNode root
   b. Kedalaman maksimum
   c. Kumpulan vertex dan face awal
   d. Kumpulan vertex dan face setelah di voxelisasi
4. OctreeNode sebagai Node tiap voxel, menyimpan
   a. vertex minimum dan maximum yang berfungsi sebagai bounding box.
   b. 8 OctreeNode children
   c. Kedalaman pohon
   d. Kumpulan sisi yang berpotongan



Gambar kubus dengan minVertex dan maxVertex sebagai bounding box

B. Algoritma

Voxel / kubus direpresentasikan sebagai 8 vertex (titik sudut kubus) dan 12 face (sisi) segitiga (6 sisi, tiap sisi dibangun dari 2 segitiga). Sebelum melakukan divide and conquer, praproses data dari file .obj:

1. Input data vertices dan faces dari file .obj ke vector, serta kedalaman maksimum (maxDepth).
2. Inisialisasi Octree

```
Vertex minV = minVertex(vertices);
Vertex maxV = maxVertex(vertices);
Octree octree(minV, maxV, maxDepth, vertices, faces);
```

```
Octree::Octree(Vertex min, Vertex max, int maxDepth,
            const std::vector<Vertex>& vertices,
            const std::vector<Face>& faces)
{
    root = createOctreeNode(min, max, 0);
    this->maxDepth = maxDepth;
    this->initialVertices = vertices;
    this->initialFaces = faces;
    for (int i = 0; i < (int)faces.size(); i++)
        root->intersectFaces.push_back(i);
}
```

Selanjutnya, lakukan Divide and Conquer

1. Divide

   Tahap ini direpresentasikan oleh prosedur

```
void Octree::divide(OctreeNode* node)
```

   Setiap node (kubus) dibagi menjadi 8. Langkah-langkah:

   a. Hitung titik tengah bounding box:

```
Vertex mid{
```

```
        (node->min.x + node->max.x) / 2,
        (node->min.y + node->max.y) / 2,
        (node->min.z + node->max.z) / 2
    };
```

b. Bagi menjadi 8 child node. Setiap child memiliki kombinasi min/max berdasarkan titik tengah

```
for (int i = 0; i < 8; i++) {
        Vertex minChild;
        Vertex maxChild;

        switch (i)
        {
            case 0:
                minChild = Vertex{node->min.x, node->min.y, node->min.z};
                maxChild = Vertex{mid.x, mid.y, mid.z};
                break;

            case 1:
                minChild = Vertex{mid.x, node->min.y, node->min.z};
                maxChild = Vertex{node->max.x, mid.y, mid.z};
                break;

            case 2:
                minChild = Vertex{node->min.x, mid.y, node->min.z};
                maxChild = Vertex{mid.x, node->max.y, mid.z};
                break;

            case 3:
                minChild = Vertex{mid.x, mid.y, node->min.z};
                maxChild = Vertex{node->max.x, node->max.y, mid.z};
                break;

            case 4:
                minChild = Vertex{node->min.x, node->min.y, mid.z};
                maxChild = Vertex{mid.x, mid.y, node->max.z};
                break;

            case 5:
                minChild = Vertex{mid.x, node->min.y, mid.z};
                maxChild = Vertex{node->max.x, mid.y, node->max.z};
                break;

            case 6:
```

```
                minChild = Vertex{node->min.x, mid.y, mid.z};
                maxChild = Vertex{mid.x, node->max.y, node->max.z};
                break;

            case 7:
                minChild = Vertex{mid.x, mid.y, mid.z};
                maxChild = Vertex{node->max.x, node->max.y, node->max.z};
                break;
        }
         node->children[i] = createOctreeNode(minChild, maxChild,
node->depth + 1);
```

c. Rekursi:

```
divide(node->children[i]);
```

Base case:

i. Sudah mencapai kedalaman maksimum, atau
ii. Tidak ada face yang relevan di node tersebut

```
if (node->depth >= maxDepth || node->intersectFaces.empty()) {

        node->isLeaf = true;

        return;

    }
```

2. Conquer

Pada setiap children, Filter face yang beririsan dengan node children berdasarkan bounding box children dari kumpulan instersectFaces parent:

```
for (int idx : node->intersectFaces) {

     if (intersect(node->children[i], initialFaces[idx]))
node->children[i]->intersectFaces.push_back(idx);

}
```

3. Combine

Tahap ini dilakukan pada

```
void Octree::combine(const OctreeNode* node)
```

Tahap ini menggabungkan hasil dari semua leaf node menjadi voxel.

Langkah-Langkah:

a. Hitung statistik node:

```
nodeCount[node->depth]++;
```

b. Jika node adalah leaf dan berisi face, konversi node menjadi voxel:

```
if (node->isLeaf && !node->intersectFaces.empty()) {

            collectVoxels(node);

            return;

    }
```

Voxel dikumpulkan menggunakan bounding box node

```
void Octree::collectVoxels(const OctreeNode* node){

    Vertex min = node->min;

    Vertex max = node->max;

    int startIdx = (int)voxelizedVertices.size();

    // 8 Vertex kubus

    voxelizedVertices.push_back({min.x, min.y, min.z}); // 0

    voxelizedVertices.push_back({max.x, min.y, min.z}); // 1

    voxelizedVertices.push_back({max.x, max.y, min.z}); // 2

    voxelizedVertices.push_back({min.x, max.y, min.z}); // 3

    voxelizedVertices.push_back({min.x, min.y, max.z}); // 4
```

```
    voxelizedVertices.push_back({max.x, min.y, max.z}); // 5
    voxelizedVertices.push_back({max.x, max.y, max.z}); // 6
    voxelizedVertices.push_back({min.x, max.y, max.z}); // 7

    // 12 sisi segitiga (6 sisi kubus)
    //bottom
    voxelizedFaces.push_back({startIdx + 0, startIdx + 1, startIdx + 2});
    voxelizedFaces.push_back({startIdx + 0, startIdx + 2, startIdx + 3});
    // top
    voxelizedFaces.push_back({startIdx + 4, startIdx + 5, startIdx + 6});
    voxelizedFaces.push_back({startIdx + 4, startIdx + 6, startIdx + 7});
    // front
    voxelizedFaces.push_back({startIdx + 0, startIdx + 1, startIdx + 5});
    voxelizedFaces.push_back({startIdx + 0, startIdx + 5, startIdx + 4});
    // right
    voxelizedFaces.push_back({startIdx + 1, startIdx + 2, startIdx + 6});
    voxelizedFaces.push_back({startIdx + 1, startIdx + 6, startIdx + 5});
    // back
    voxelizedFaces.push_back({startIdx + 2, startIdx + 3, startIdx + 7});
    voxelizedFaces.push_back({startIdx + 2, startIdx + 7, startIdx + 6});
    // left
    voxelizedFaces.push_back({startIdx + 3, startIdx + 0, startIdx + 4});
    voxelizedFaces.push_back({startIdx + 3, startIdx + 4, startIdx + 7});
}
```

c. Rekursi ke child:

```
for (int i = 0; i < 8; i++) {

            if (node->children[i])

                combine(node->children[i]);

    }
```

C. Kompleksitas Algoritma

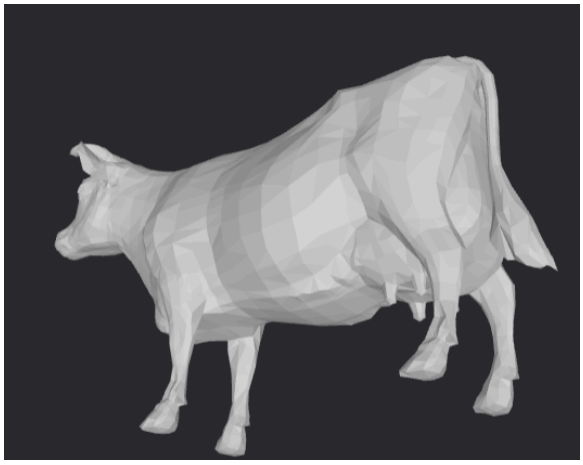
Karena rekursi divide dipanggil 8 kali untuk setiap node, dan setiap rekursi memanggil divide dan loop sebanyak faces hingga mencapai maxDepth, maka Kompleksitas waktunya adalah

$$O(8^D \cdot F)$$

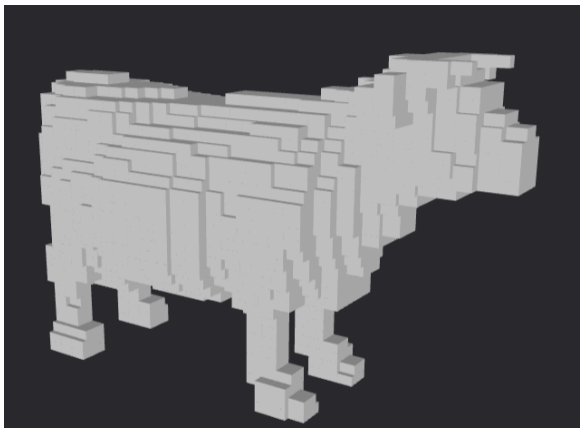
D = Kedalaman maksimum
F = Jumlah Face

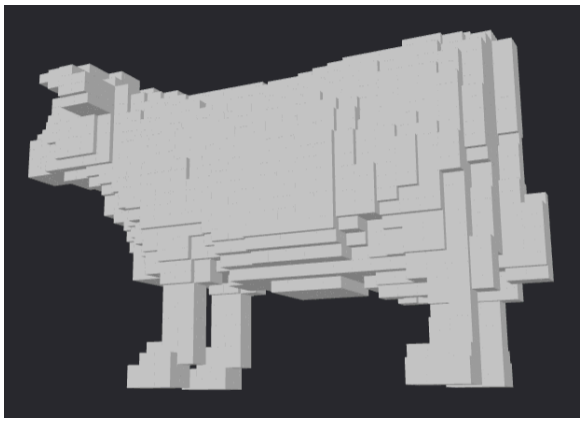
Hasil Pengujian

1. Cow.obj
   Objek 3D Awal:



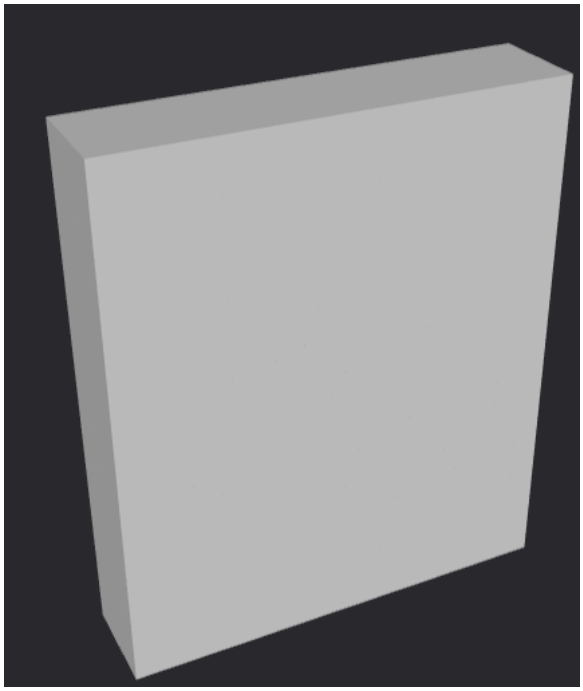
Objek setelah di voxelisasi dengan maxDepth = 5:



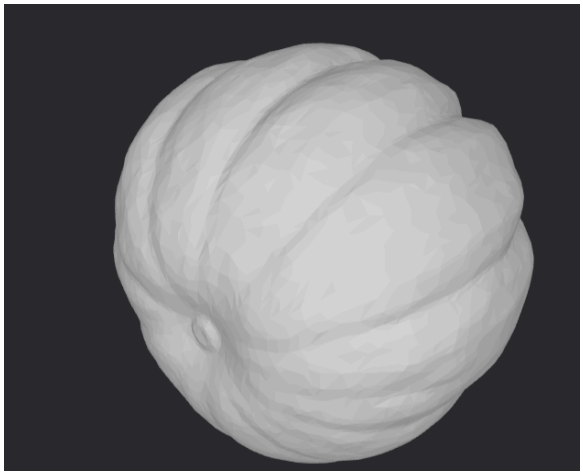


2. Line.obj
   Objek 3D Awal:

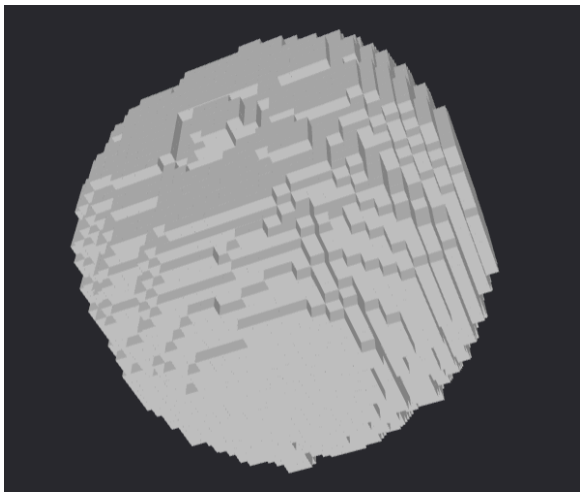
Objek setelah di voxelisasi dengan maxDepth = 5:



3. Pumpkin.obj
   Objek 3D Awal:



Objek setelah di voxelisasi dengan maxDepth = 5:



4. Teapot.obj
   Objek 3D Awal:

Objek setelah di voxelisasi dengan maxDepth = 5:

Program menolak input karena mengandung line dengan format tidak valid (bukan v x y z atau f i j k)

Lampiran

Pranala repositori: https://github.com/wzlyy/Tucil2_13524054

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan *concurrency*	✓	
7	[Bonus] Program dapat menampilkan visualisasi .obj		✓

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Wildan Abdurrahman Ghazali